



UNIVERSIDAD TECNOLÓGICA DE PUEBLA

INGENIERÍA EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

APLICACIÓN WEB PROGRESIVA

REPORTE — SUBPRODUCTO NO. 7
(ACTIVIDAD CIERRE)

**ASIGNATURA: APLICACIONES WEB
PROGRESIVAS**

DOCENTE: MATHER XÓCHITL MENDOZA
PÍSCIL

ALUMNO: PÉREZ ALCÁNTARA ALEJANDRO
10° “D”

FECHA: 29 DE SEPTIEMBRE DE 2026

ÍNDICE

1. Introducción.....	3
2. Definición de una Aplicación Web Progresiva (PWA)	3
2.1 Tecnologías Clave que la Definen	3
2.2 Breve Historia y Evolución	4
3. Ilustración: ¿Cómo funciona una Aplicación Web Progresiva?	4
3.1 Ciclo de Vida del Service Worker	6
4. Características Principales	7
5. Ventajas y Desventajas	8
6. Herramientas de Desarrollo.....	10
Lighthouse	10
Workbox	11
PWA Builder	11
7. Ejemplos Reales de Aplicaciones Web Progresivas.....	12
7.1 Starbucks.....	12
7.2 X (Twitter Lite)	13
7.3 Pinterest	13
7.4 AliExpress	13
7.5 Uber	13
7.6 Forbes	13
8. Evidencias de Identificación de Características	14
8.1 Evidencia 1 — Web App Manifest e Instalable	14
8.2 Evidencia 2 — Service Worker y Funcionamiento sin Conexión	15
8.3 Evidencia 3 — Diseño Responsivo	16
9. Casos de Uso Recomendados	18
10. Comparativa: PWA vs. los Enfoques Documentados en el Curso.....	19
11. Lista de Cotejo.....	20
12. Conclusión	21
Referencias.....	22

1. Introducción

A lo largo de esta materia he documentado seis enfoques de desarrollo de software: en el Subproducto No. 1 el desarrollo nativo (mi reloj inteligente en Wear OS); en el No. 2 el desarrollo web (MediCore Professional System); en el No. 3 el desarrollo de escritorio (TaskDesk); en el No. 4 el desarrollo móvil (PediatriTrack Global); en el No. 5 el desarrollo multiplataforma (ExpenseTrack Lite); y en el No. 6 el desarrollo orientado a servicios (HabitPulse). Esta séptima actividad, de cierre, aborda un enfoque distinto a los anteriores: la Aplicación Web Progresiva o PWA, que no es un tipo de arquitectura nueva sino una forma de llevar una aplicación web tradicional como la que documenté en el Subproducto No. 2 a comportarse como una aplicación instalable, con capacidades que antes solo tenían las apps nativas.

Siguiendo la instrucción de esta actividad, exploré el material de lectura sobre Aplicaciones Web Progresivas y complementé esa investigación con documentación oficial y casos de estudio reales de empresas que ya las usan en producción. A lo largo del documento defino qué es una PWA, listo sus características principales, sus ventajas y desventajas, documento las herramientas que existen para desarrollarlas, presento ejemplos reales con resultados medibles, y cierro comparando este enfoque contra los seis que ya documenté durante el curso.

2. Definición de una Aplicación Web Progresiva (PWA)

Una Aplicación Web Progresiva (Progressive Web App o PWA) es una aplicación construida con las tecnologías estándar de la web HTML, CSS y JavaScript que, gracias a un conjunto específico de APIs del navegador, puede instalarse en el dispositivo del usuario, funcionar sin conexión a internet y comportarse visualmente como una aplicación nativa, sin dejar de ser, por dentro, un sitio web servido desde un servidor por HTTPS. El término fue introducido por Google en 2015 para describir sitios web que "progresan" en capacidades según lo que soporte el navegador del usuario: en un navegador antiguo funcionan como una página web normal, y en uno moderno se pueden instalar, notificar y usar sin internet.

La diferencia central frente a los enfoques que ya documenté es que una PWA no es una categoría de tecnología aparte (como sí lo son Electron para multiplataforma o React Native para móvil), sino un conjunto de capacidades adicionales que se le agregan a una aplicación web existente. Esto conecta directamente con el Subproducto No. 2: MediCore Professional System, tal como lo documenté entonces, es un candidato directo para convertirse en PWA, ya que basta con agregarle un archivo de manifiesto y un service worker sin reescribir su lógica ni su interfaz.

2.1 Tecnologías Clave que la Definen

De acuerdo con la documentación oficial (MDN, web.dev) y el material consultado, tres elementos técnicos son los que convierten un sitio web común en una PWA:

- **HTTPS obligatorio:** el sitio debe servirse por una conexión segura (HTTPS), requisito indispensable para que el navegador permita registrar un service worker y confíe en instalar la aplicación.
- **Web App Manifest (manifest.json):** un archivo de metadatos en formato JSON que declara el nombre de la app, sus íconos (típicamente en 192px y 512px), el color del tema, la pantalla de inicio y el modo de visualización (standalone, fullscreen o minimal-ui), y que es lo que permite "Agregar a pantalla de inicio".

- **Service Worker (sw.js):** un script que el navegador ejecuta en segundo plano, independiente de la página, que intercepta las peticiones de red, cachea archivos y datos, habilita el funcionamiento sin conexión y gestiona las notificaciones push y la sincronización en segundo plano.

Estos tres elementos no funcionan de forma aislada: el navegador solo permite registrar un service worker si el sitio se sirve por HTTPS, y solo ofrece la opción de "Instalar" o "Agregar a pantalla de inicio" si detecta un manifest.json válido junto con un service worker activo. Es decir, los tres requisitos son necesarios en conjunto, no opcionales por separado: quitar cualquiera de los tres hace que el sitio siga siendo una página web normal, pero deja de calificar como PWA.

2.2 Breve Historia y Evolución

El concepto de Aplicación Web Progresiva lo acuñaron Frances Berriman y Alex Russell, ingeniera de Google, en 2015, aunque las tecnologías que lo hacen posible en particular la API de Service Worker ya se venían discutiendo desde antes dentro del grupo de trabajo de aplicaciones web fuera de línea de W3C. Google impulsó el concepto como una respuesta a un problema muy concreto: las apps nativas ofrecían una mejor experiencia que la web (instalación, funcionamiento sin conexión, notificaciones), pero desarrollar y mantener una app nativa por cada sistema operativo era costoso, mientras que un sitio web, aunque barato de mantener, perdía usuarios frente a apps más rápidas y confiables.

Entre 2015 y 2017, Chrome fue el primer navegador en implementar de forma completa el soporte a service workers y al Web App Manifest, seguido por Firefox y Edge. Apple fue, y sigue siendo, el navegador más rezagado en el soporte completo: Safari agregó soporte básico a service workers hasta 2018, y durante años limitó funciones como las notificaciones push en iOS, que no se habilitaron de forma general hasta 2023 con iOS 16.4. Esa diferencia de soporte entre navegadores explica buena parte de las desventajas que documento en la sección 5 de este reporte.

En años recientes, la adopción de PWA ha crecido de la mano de frameworks modernos (React, Vue, Angular) que integran el soporte a PWA como una opción de configuración más, y de herramientas como las que documento en la sección 6, que automatizan la generación del manifiesto y del service worker. Esto ha hecho que convertir un sitio web existente en una PWA sea, en 2026, una tarea de horas y no de semanas, lo cual explica por qué empresas grandes (documentadas en la sección 7) la adoptaron como una alternativa más barata que mantener una app nativa completa.

3. Ilustración: ¿Cómo funciona una Aplicación Web Progresiva?

El siguiente diagrama ilustra cómo estos dos archivos (manifest.json y sw.js), agregados a un sitio web servido por HTTPS, producen las dos capacidades que distinguen a una PWA de un sitio web tradicional: la posibilidad de instalarse en la pantalla de inicio del dispositivo, y la posibilidad de seguir funcionando sin conexión a internet o con una red lenta.

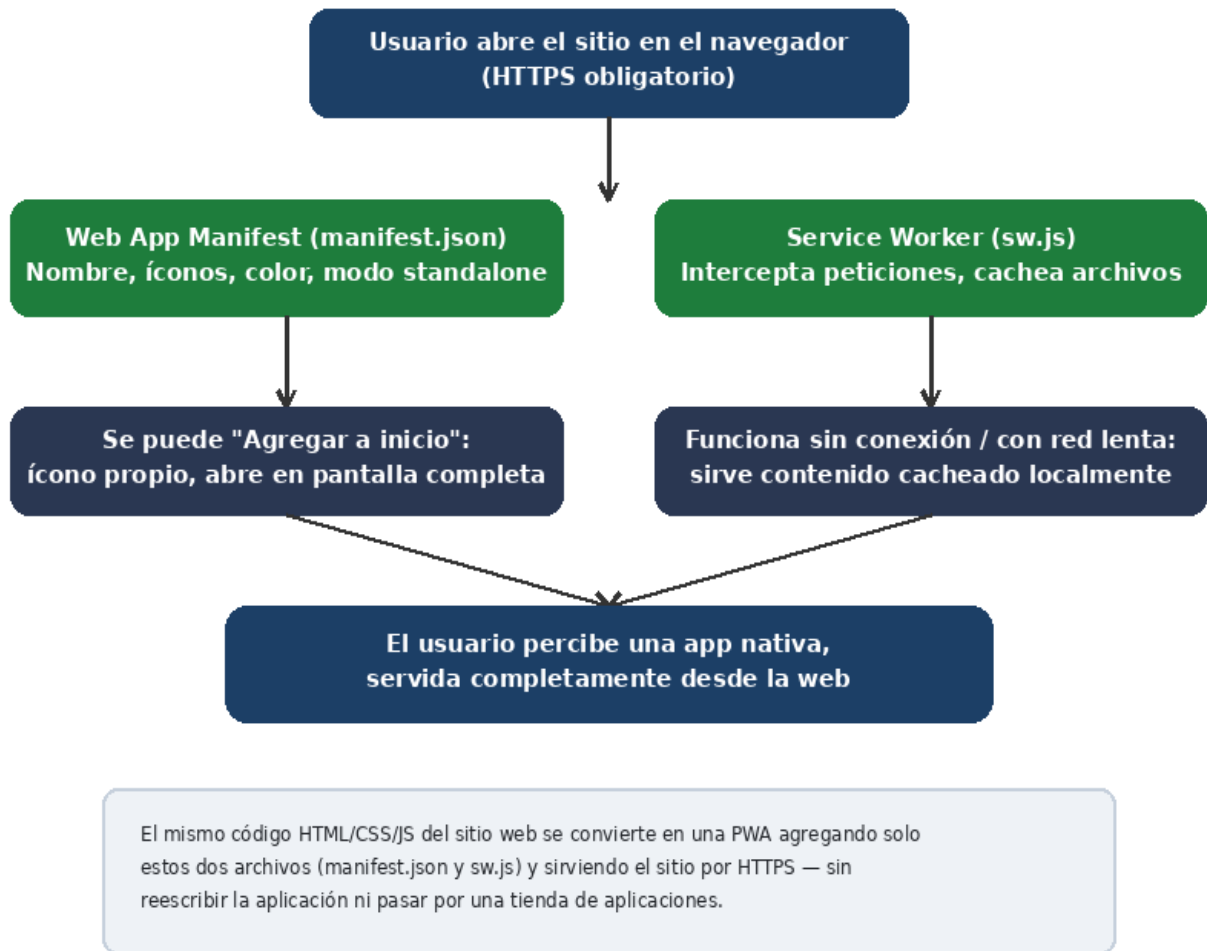


Figura 1. Flujo de funcionamiento de una PWA: el Web App Manifest habilita la instalación y el Service Worker habilita el funcionamiento sin conexión, ambos sobre el mismo sitio web servido por HTTPS.

3.1 Ciclo de Vida del Service Worker

El service worker es el elemento más complejo de una PWA porque, a diferencia del resto del código de la página, no se ejecuta junto con ella sino en un proceso aparte que el navegador administra por su cuenta, con un ciclo de vida propio de seis fases:

FASE	QUÉ OCURRE	EVENTO EN EL CÓDIGO
1. REGISTRO	La página le pide al navegador que registre el archivo sw.js como su service worker	navigator.serviceWorker.register('/sw.js')
2. INSTALACIÓN	El service worker se descarga y cachea los archivos iniciales que la app necesita para funcionar sin conexión	evento install
3. ACTIVACIÓN	El nuevo service worker reemplaza al anterior (si existía) y limpia las versiones de caché que ya quedaron obsoletas	evento activate
4. INTERCEPCIÓN DE PETICIONES	Cada vez que la página pide un archivo o dato, el service worker decide si lo sirve desde caché, desde la red, o combina ambas	evento fetch
5. NOTIFICACIONES PUSH	El service worker recibe un mensaje del servidor, aunque la app esté cerrada, y muestra una notificación al usuario	evento push
6. SINCRONIZACIÓN EN SEGUNDO PLANO	Cuando vuelve la conexión, el service worker envía al servidor los datos que el usuario generó estando sin internet	evento sync

Entender este ciclo es importante porque explica por qué, la primera vez que un usuario visita una PWA, está todavía se comporta como un sitio web normal: el service worker necesita completar las fases de registro, instalación y activación antes de poder interceptar peticiones. Solo a partir de la segunda visita, cuando el service worker ya está activo, el usuario percibe las ventajas de velocidad y funcionamiento sin conexión que documenta en las siguientes secciones.

4. Características Principales

El material de lectura consultado y la documentación oficial de Google (web.dev) coinciden en un conjunto de diez características que, en conjunto, distinguen a una PWA de un sitio web tradicional. Las desarrollo a continuación con el detalle técnico de cada una:

- **Multiplataforma:** un mismo código HTML, CSS y JavaScript corre, desde el navegador, en Android, iOS, Windows, macOS y Linux, sin compilar versiones distintas por sistema operativo como sí exige una app nativa.

Esto es posible porque la PWA no depende del sistema operativo sino del motor del navegador (Chrome, Edge, Safari, Firefox), que está disponible en prácticamente cualquier dispositivo moderno. Es la misma razón por la que documenté en el Subproducto No. 5 que un enfoque multiplataforma como Electron reduce el mantenimiento frente a desarrollar nativo por plataforma; una PWA lleva esa misma idea un paso más allá, porque ni siquiera requiere empaquetar ni distribuir un instalador distinto por sistema operativo.

- **Instalable:** el usuario puede agregarla a la pantalla de inicio de su dispositivo desde el propio navegador, sin pasar por una tienda de aplicaciones, y se abre con su propio ícono en una ventana sin la barra de direcciones del navegador.

Esta instalación ocurre mediante el evento `beforeinstallprompt`, que el navegador dispara cuando detecta que el sitio cumple los requisitos técnicos (manifiesto válido, service worker registrado, HTTPS). El desarrollador puede capturar este evento para mostrar su propio botón de "Instalar app" en el momento que considere más adecuado, en vez de depender únicamente del ícono que agrega el navegador en la barra de direcciones.

- **Funciona sin conexión o con red inestable:** el service worker sirve contenido cacheado cuando no hay internet, o cuando la conexión es muy lenta, en vez de mostrar una pantalla de error.

Esto se logra con distintas estrategias de cacheo que el desarrollador elige según el tipo de contenido: `cache-first` (prioriza lo guardado localmente y solo va a la red si no existe en caché, ideal para íconos y estilos que casi nunca cambian), `network-first` (intenta la red primero y usa el caché solo si falla, ideal para datos que deben estar actualizados) y `stale-while-revalidate` (sirve el caché de inmediato mientras actualiza esa copia en segundo plano para la próxima vez).

- **Segura por diseño:** al requerir HTTPS de forma obligatoria, todo el tráfico entre el usuario y el servidor va cifrado desde el primer requisito técnico.
- **Responsiva:** al estar construida con HTML, CSS y JavaScript estándar, se adapta al tamaño de pantalla de cualquier dispositivo, de un celular a una computadora de escritorio, con el mismo código.
- **Actualizable de forma transparente:** como sigue siendo un sitio web, cada vez que el usuario la abre con conexión, el service worker puede descargar la versión más reciente, sin que el usuario tenga que instalar una actualización manualmente.

El navegador revisa en segundo plano si el archivo `sw.js` cambió desde la última visita; si detecta una versión nueva, la descarga e instala en paralelo, y la activa la siguiente vez que el usuario recarga la aplicación. Esto elimina por completo el paso de "hay una actualización disponible, ¿deseas instalarla?" que sí existe en las apps nativas distribuidas por tienda.

- **Descubrible y enlazable:** conserva una URL como cualquier página web, por lo que puede indexarse en buscadores y compartirse con un enlace directo, algo que una app nativa no ofrece de forma nativa.
- **Capaz de enviar notificaciones push:** mediante el service worker, puede recibir y mostrar notificaciones incluso cuando la aplicación no está abierta, de forma similar a una app nativa.

Esta capacidad depende de la API Push del navegador combinada con un servicio de mensajería (por ejemplo, Firebase Cloud Messaging), y requiere que el usuario otorgue permiso explícito la primera vez. Como mencioné en la sección 2.2, en dispositivos Apple esta función solo está disponible de forma completa desde iOS 16.4, lo que durante años limitó su adopción en ese ecosistema.

- **Insignias en el ícono (Badging API):** una vez instalada, la PWA puede mostrar un pequeño contador o punto sobre su propio ícono igual que el número de mensajes sin leer en WhatsApp o el correo para avisar al usuario de pendientes o notificaciones sin necesidad de abrir la aplicación.

Esto se implementa con la Badging API del navegador (`navigator.setAppBadge(numero)` para mostrar un número, y `navigator.clearAppBadge()` para quitarlo), y solo funciona una vez que la aplicación ya está instalada como PWA; en un sitio web abierto en una pestaña normal del navegador no aparece esta insignia. Su soporte, al igual que las notificaciones push, es más limitado en Safari/iOS que en Chrome/Edge sobre Windows y Android.

- **Documentable mediante las herramientas del navegador:** a diferencia de una app nativa compilada, el código de una PWA (HTML, CSS y JavaScript) puede inspeccionarse directamente desde el navegador con el botón "Inspeccionar" (DevTools, tecla F12), sin necesidad de descompilar nada.

Esto permite abrir la pestaña Elements para ver la estructura HTML y los estilos aplicados en tiempo real, la pestaña Application para revisar el manifest.json, el service worker registrado y el contenido cacheado, y la pestaña Network para observar cada petición que hace la aplicación. Para efectos de documentación y auditoría técnica, esta es una ventaja concreta frente a una app nativa: cualquier persona con el navegador, sin herramientas adicionales, puede verificar que el manifiesto, el service worker y el resto del código funcionan tal como se describe en este reporte.

5. Ventajas y Desventajas

A partir de las características anteriores y de los casos de estudio consultados, resumo las principales ventajas y desventajas de este enfoque:

VENTAJAS	DESVENTAJAS
Un solo código para web, escritorio y móvil, sin mantener versiones nativas separadas por plataforma.	En iOS/Safari el soporte histórico ha sido más limitado que en Android/Chrome, aunque ha mejorado con las versiones recientes.
No requiere pasar por la revisión ni las políticas de una tienda de aplicaciones para publicarse o actualizarse.	El acceso a algunas APIs de hardware avanzadas (Bluetooth, NFC, sensores específicos) sigue siendo limitado frente a una app nativa.
Es mucho más ligera que una app nativa equivalente: Twitter Lite pesa menos del 3% del tamaño de sus apps nativas.	Es menos descubrible que una app en una tienda, ya que no aparece en App Store ni Google Play a menos que se distribuya también ahí.

VENTAJAS	DESVENTAJAS
Funciona sin conexión o con conexión lenta gracias al service worker, ampliando el alcance a mercados con internet limitado.	El cacheo agresivo del service worker puede consumir almacenamiento del dispositivo si no se gestiona correctamente.
Se actualiza de forma transparente para el usuario, sin fricción de descargar una nueva versión desde una tienda.	Requiere que el equipo de desarrollo entienda y mantenga correctamente el ciclo de vida del service worker, que puede ser complejo de depurar.

Viendo estas cinco parejas de ventajas y desventajas en conjunto, noto un patrón: casi todas las desventajas están relacionadas con el nivel de soporte que le da cada navegador o sistema operativo a la tecnología, no con una limitación de diseño del concepto de PWA en sí. Apple, por ejemplo, ha ido cerrando esa brecha con el tiempo (notificaciones push desde iOS 16.4, mejor soporte de service workers en versiones recientes de Safari), lo que sugiere que varias de estas desventajas se seguirán reduciendo en los próximos años, de la misma forma en que ya se redujeron entre 2015 y 2023.

La otra cara de la moneda es que las ventajas de una PWA dependen directamente de qué tan bien se implementen las estrategias de cacheo del service worker: una PWA mal configurada, que cachea contenido desactualizado o que no gestiona correctamente el ciclo de vida descrito en la sección 3.1, puede terminar ofreciendo una peor experiencia que un sitio web tradicional sin ninguna de estas capacidades. Por eso las herramientas de la siguiente sección en particular Lighthouse y Workbox son tan relevantes: automatizan las partes de la implementación donde es más fácil cometer errores.

6. Herramientas de Desarrollo

Estas son las herramientas más mencionadas en el material consultado y en la documentación oficial para construir y validar una PWA:

HERRAMIENTA	TIPO	PARA QUÉ SE USA
Lighthouse	Auditor (Chrome DevTools)	Analiza un sitio y genera un puntaje de "PWA", señalando qué requisitos faltan (manifiesto, HTTPS, service worker, rendimiento, accesibilidad)
Workbox	Librería (Google)	Genera y administra el service worker con estrategias de cacheo ya probadas (cache-first, network-first, etc.), en vez de escribirlo a mano
PWA Builder	Generador (Microsoft)	Analiza un sitio existente, genera su manifest.json y service worker, y empaqueta la PWA para publicarla en Microsoft Store, Google Play o App Store
vite-plugin-pwa / CRA PWA template	Plugin de build	Integra automáticamente el manifiesto y el service worker al compilar un proyecto hecho con Vite o Create React App
Chrome DevTools — pestaña Application	Herramienta del navegador	Inspecciona el manifiesto, el estado del service worker y el contenido cacheado; simula estar sin conexión para probar el comportamiento offline
Web App Manifest Generator	Generador en línea	Crea el archivo manifest.json y los íconos en los tamaños requeridos a partir de un logotipo, sin escribirlo manualmente

En conjunto, estas herramientas cubren las tres etapas del desarrollo de una PWA: generar los archivos técnicos (manifiesto y service worker) sin escribirlos a mano con Workbox, vite-plugin-pwa o PWA Builder; verificar que cumplan los requisitos con Lighthouse; y depurar su comportamiento en tiempo real, incluyendo el modo sin conexión, desde la pestaña Application de Chrome DevTools. Detallo a continuación las tres que considero más relevantes para un desarrollador que recién comienza con PWA:

Lighthouse

Es la herramienta de referencia para saber qué tan cerca está un sitio de calificar como PWA. Viene integrada directamente en Chrome DevTools (pestaña Lighthouse) y también existe como paquete de línea de comandos y como servicio en línea (PageSpeed Insights). Al ejecutarla sobre una URL, genera un reporte con un puntaje de 0 a 100 en la categoría "Progressive Web App" además de rendimiento, accesibilidad y buenas prácticas y, más importante que el puntaje, una lista concreta de qué falta: si no hay manifest.json, si el service worker no responde sin conexión, si faltan íconos en algún tamaño, o si el sitio no se sirve por HTTPS.

Workbox

Es la librería que Google recomienda oficialmente para escribir el service worker, en vez de programarlo desde cero. Escribir a mano la lógica de qué cachear, cuándo actualizar el caché y qué estrategia usar para cada tipo de archivo es propenso a errores sutiles (como servir contenido desactualizado indefinidamente). Workbox resuelve esto con funciones ya probadas para las estrategias de cacheo que mencioné en la sección 4 (cache-first, network-first, stale-while-revalidate), y se integra tanto de forma manual como a través de plugins de build como Workbox Webpack Plugin.

PWA Builder

Es una herramienta de Microsoft pensada para el caso más común entre desarrolladores que ya tienen un sitio web funcionando y quieren convertirlo en PWA sin reescribirlo. Basta con ingresar la URL del sitio y la herramienta lo analiza, genera automáticamente el manifest.json y un service worker básico, y a diferencia de las otras herramientas de esta tabla— también empaqueta el resultado para poder publicarlo directamente en Microsoft Store, Google Play (mediante Trusted Web Activity) o, con más limitaciones, en la App Store de Apple.

7. Ejemplos Reales de Aplicaciones Web Progresivas

A diferencia de los subproductos anteriores, en esta actividad no construí una aplicación propia, ya que la actividad pide explorar el material de lectura y documentar el panorama general de las PWA, no ilustrarlo con un ejemplo propio. En su lugar, documento aquí seis casos reales y públicos de empresas que migraron a una PWA, con su URL y los resultados que reportaron:

EMPRESA / PWA	URL	RESULTADO REPORTADO	DETALLE
Starbucks	order.starbucks.com	Duplicó los usuarios activos diarios en web	Su PWA pesa 233 KB frente a los 148 MB de su app nativa para iOS (99.84% más ligera) e incluye carrito funcional sin conexión
X (Twitter Lite)	x.com (antes mobile.twitter.com)	+65% páginas por sesión, +75% tweets enviados	Usa menos del 3% de los datos que consumen sus apps nativas, y carga en 5 segundos sobre una red 3G
Pinterest	pinterest.com	+40% tiempo de uso, +44% ingresos por publicidad	Redujo su tiempo de carga en 3G de 23 a menos de 5 segundos al migrar a una PWA
AliExpress	m.aliexpress.com	+104% conversión en usuarios nuevos, +82% en iOS	El doble de páginas vistas por sesión gracias a la carga rápida en conexiones lentas
Uber	m.uber.com	Funcional incluso en redes 2G	Carga inicial de solo 50 KB, pensada para dispositivos de gama baja en mercados con internet limitado
Forbes	forbes.com	+50% sesiones completas, 3x más scroll	La velocidad de carga de la PWA incrementó directamente el tiempo que los lectores pasan en los artículos

El patrón que se repite en los seis casos es el mismo: la ganancia principal viene de la velocidad de carga y del menor peso de la aplicación frente a su versión nativa equivalente, lo cual se traduce directamente en más sesiones, más conversión y más tiempo de uso, especialmente en mercados con conexiones de red más lentas o dispositivos de gama baja. Detallo a continuación el contexto de cada caso:

7.1 Starbucks

Starbucks lanzó su PWA (order.starbucks.com) pensando específicamente en los usuarios que no querían instalar la app nativa completa solo para ver el menú o hacer un pedido ocasional. El resultado más citado es que duplicó el número de usuarios activos diarios en la versión web frente a la app nativa, gracias en buena parte a que su PWA pesa apenas 233 KB frente a los 148 MB de la app nativa para iOS una diferencia de tamaño de más del 99% y a que incluye un carrito de compras que sigue funcionando, aunque el usuario pierda la conexión momentáneamente, algo común dentro de una tienda física.

7.2 X (Twitter Lite)

Twitter Lite ahora parte de X es uno de los casos más documentados en la literatura sobre PWA porque fue de los primeros en publicar cifras concretas. Reportó un incremento del 65% en páginas vistas por sesión y del 75% en tuits enviados tras el lanzamiento de la PWA, atribuido a que la aplicación usa menos del 3% de los datos móviles que consumen sus apps nativas para Android e iOS, y carga en aproximadamente 5 segundos incluso sobre una red 3G, un escenario común en varios de los mercados donde la plataforma creció más rápido.

7.3 Pinterest

Pinterest rediseñó por completo su sitio móvil como PWA después de notar que su versión web anterior tardaba hasta 23 segundos en cargar sobre una red 3G, tiempo en el que la mayoría de los usuarios abandonaba la página antes de que terminara de cargar. Tras el cambio, ese tiempo de carga bajó a menos de 5 segundos, lo que se tradujo en un 40% más de tiempo que los usuarios pasan en el sitio y un 44% más de ingresos por publicidad, ya que los anuncios ahora sí llegan a cargar y a ser vistos.

7.4 AliExpress

AliExpress migró su sitio de compras a una PWA con el objetivo específico de mejorar la conversión en mercados con conexiones más lentas, un factor crítico para una tienda que envía productos a nivel internacional. Reportó un incremento del 104% en la tasa de conversión entre usuarios nuevos y del 82% específicamente en usuarios de iOS, además de que los usuarios de la PWA navegan, en promedio, el doble de páginas por sesión que en la versión web anterior.

7.5 Uber

La versión PWA de Uber (m.uber.com) está diseñada explícitamente para el escenario más exigente: pedir un viaje desde un dispositivo de gama baja con una conexión 2G. Su carga inicial pesa apenas 50 KB, y aun así conserva las funciones esenciales del servicio —ubicación, seguimiento del viaje en tiempo real y notificaciones, lo que permite que el servicio sea utilizable en mercados donde instalar y actualizar la app nativa completa de Uber sería poco práctico por el costo de los datos móviles o el espacio de almacenamiento del teléfono.

7.6 Forbes

Forbes migró su sitio de noticias a una PWA principalmente por motivos de retención de lectores: reportó un incremento del 50% en la tasa de sesiones completadas (el usuario termina de leer el artículo en vez de abandonarlo a medio camino) y hasta 3 veces más profundidad de scroll por sesión, dos métricas que la publicación atribuye directamente a que los artículos cargan más rápido y de forma más confiable que en su versión web anterior.

8. Evidencias de Identificación de Características

Para no quedarme solo en la descripción teórica de la sección 4, capturé evidencia real de tres características de PWA identificadas en vivo sobre Squoosh (squoosh.app), la herramienta de compresión de imágenes que el propio equipo de Chrome de Google construyó como PWA de referencia. La elegí porque es pública, no requiere inicio de sesión, y su manifiesto y service worker son completamente inspeccionables desde el navegador, lo que permite documentar cada característica con una captura verificable en vez de solo describirla.

8.1 Evidencia 1 — Web App Manifest e Instalable

Impresión con formato estilístico ☐

```
{
  "name": "Squoosh",
  "short_name": "Squoosh",
  "start_url": "?utm_medium=PWA&utm_source=launcher",
  "display": "standalone",
  "orientation": "any",
  "background_color": "#fff",
  "theme_color": "#ff3385",
  "icons": [
    {
      "src": "/c/icon-large-cb438cac.png",
      "type": "image/png",
      "sizes": "1024x1024"
    },
    {
      "src": "/c/icon-large-maskable-c2078ced.png",
      "type": "image/png",
      "sizes": "1024x1024",
      "purpose": "maskable"
    }
  ],
  "description": "Compress and compare images with different codecs, right in your browser.",
  "lang": "en",
  "categories": [
    "photo",
    "productivity",
    "utilities"
  ],
  "screenshots": [
    {
      "src": "/c/screenshot1-0ff68546.png",
      "type": "image/png",
      "sizes": "540x720",
      "form_factor": "narrow"
    },
    {
      "src": "/c/screenshot2-1f78c4db.jpg",
      "type": "image/jpeg",
      "sizes": "540x720",
      "form_factor": "narrow"
    },
    {
      "src": "/c/screenshot3-cl02216.jpg",
      "type": "image/jpeg",
      "sizes": "540x720",
      "form_factor": "narrow"
    },
    {
      "src": "/c/screenshot4-3a706c3c.png",
      "type": "image/png",
      "sizes": "1024x593",
      "form_factor": "wide"
    },
    {
      "src": "/c/screenshot5-ea50826f.jpg",
      "type": "image/jpeg",
      "sizes": "1024x593",
      "form_factor": "wide"
    },
    {
      "src": "/c/screenshot6-0168d284.jpg",
      "type": "image/jpeg",
      "sizes": "1024x593",
      "form_factor": "wide"
    }
  ],
  "share_target": {
    "action": "?utm_medium=PWA&utm_source=share-target&share-target",
    "method": "POST",
    "enctype": "multipart/form-data",
    "params": {
      "files": [
        {
          "name": "file",
          "accept": ["image/*"]
        }
      ]
    }
  }
}
```

Figura 2. Contenido real del archivo manifest.json de squoosh.app, obtenido navegando directamente a squoosh.app/manifest.json.

Esta captura es la evidencia directa de la característica Instalable descrita en la sección 4: el manifiesto declara "name": "Squoosh", "display": "standalone" (se abre en su propia ventana, sin la barra del navegador) y varios "icons", incluyendo uno con "purpose": "maskable" para que el ícono se recorte correctamente en cualquier forma de ícono del sistema operativo. Sin estos tres campos como mínimo (name, icons, display), el navegador no ofrece la opción de instalar el sitio.

8.2 Evidencia 2 — Service Worker y Funcionamiento sin Conexión



Figura 3. Aviso "Ready to work offline" mostrado por squoosh.app la primera vez que el service worker terminó de cachear los archivos necesarios para funcionar sin conexión.

Esta captura es la evidencia directa de la característica Funciona sin conexión o con red inestable descrita en la sección 4: el mensaje que Squoosh muestra en la parte inferior de la pantalla confirma, en tiempo real y sin que yo lo simule, que su service worker terminó de instalarse y cachear localmente lo necesario para que la aplicación siga funcionando, aunque se pierda la conexión a internet.

8.3 Evidencia 3 — Diseño Responsivo



Figura 4. squoosh.app en un viewport de escritorio (527×600 px): el círculo de carga y el logo se acomodan centrados con espacio horizontal amplio.



Figura 5. squoosh.app en un viewport móvil emulado (375×812 px, mismo sitio y misma sesión): el mismo círculo de carga y la fila de ejemplos se reacomodan en una sola columna vertical sin recortarse ni requerir scroll horizontal.

Estas dos capturas, tomadas sobre la misma URL en la misma sesión del navegador, son la evidencia directa de la característica Responsiva descrita en la sección 4: el mismo HTML, CSS y JavaScript se reorganiza automáticamente según el ancho de pantalla disponible, sin que exista una versión de escritorio y otra versión móvil separadas.

9. Casos de Uso Recomendados

A partir de las ventajas y desventajas de la sección 5 y de los patrones que se repiten en los ejemplos reales de la sección 7, resumo en qué escenarios el material consultado sugiere que una PWA es la opción más adecuada, y en cuáles conviene más alguno de los otros enfoques que documenté en el curso:

ESCENARIO	¿CONVIENE UNA PWA?	POR QUÉ
Ecommerce o catálogo de productos	Sí, es de los casos más favorables	Carga rápida y funcionamiento sin conexión se traducen directamente en más conversión, como muestran los casos de AliExpress y Starbucks
Medio de noticias o blog	Sí	La velocidad de carga impacta directamente el tiempo de lectura y el scroll, como en el caso de Forbes
Red social o mensajería	Sí, con matices	Funciona bien para reducir el tamaño de la app (Twitter Lite), pero las notificaciones push en iOS solo están completas desde 2023
App bancaria o de pagos	Depende	El acceso limitado a APIs de seguridad avanzadas del dispositivo (huella digital, enclave seguro) puede ser una limitante frente a una app nativa
Juego con gráficos 3D exigentes	No es lo más recomendable	El rendimiento de una PWA para cargas gráficas intensivas suele ser menor al de una app nativa que accede directamente al hardware
App interna de una empresa (uso conocido y controlado)	Sí	Evita pasar por revisión de tienda para cada actualización, algo valioso cuando los cambios son frecuentes y el público es interno

La regla general que se desprende de esta tabla es que una PWA rinde mejor cuanto más depende el negocio de que el contenido cargue rápido y esté disponible con conexiones inestables, y rinde peor cuanto más depende la aplicación de acceso profundo al hardware del dispositivo o de un rendimiento gráfico exigente que solo una app nativa puede ofrecer de forma consistente.

10. Comparativa: PWA vs. los Enfoques Documentados en el Curso

Para cerrar el panorama completo de esta materia, comparo aquí el enfoque de Aplicación Web Progresiva frente a los seis que ya documenté: móvil (Subproducto No. 4), web tradicional (Subproducto No. 2), escritorio nativo (Subproducto No. 3), multiplataforma (Subproducto No. 5) y orientado a servicios (Subproducto No. 6):

CRITERIO	PWA	MÓVIL	WEB TRAD.	ESCRITORIO	MULTIPLATAFORMA	ORIENTADA A SERVICIOS
Lenguaje	HTML, CSS, JS	JS/TS (React Native), Kotlin/Swift	HTML, CSS, JS (+ backend)	Python, C#, C++, Java	HTML, CSS, JS + Node.js	Node.js/Express + cliente
Instalación	Sí, desde el navegador, sin tienda	Sí, desde tienda de apps	No requiere	Sí (o runtime)	Sí, instalador por SO	No para el cliente web
Funciona sin internet	Sí (service worker)	Sí, parcialmente	Muy limitado	Sí	Sí	No
Tamaño típico	Muy ligero (cientos de KB)	Medio-alto	Ligero	Medio-alto	Alto (runtime embebido)	Depende del servicio
Distribución	URL directa o tiendas opcionales	Tienda de apps obligatoria	URL directa	Instalador o tienda	Instalador por SO	Repositorio / despliegue propio
Ejemplo	Starbucks, Twitter Lite, Pinterest	PediatricTrack Global	MediCore	TaskDesk	ExpenseTrack Lite	HabitPulse

La PWA ocupa un lugar particular en esta comparativa: no es un enfoque que compita con los demás, sino una capa de capacidades que se le puede agregar a una aplicación web tradicional (como MediCore) para que se acerque a lo que ofrecen los enfoques móvil, de escritorio o multiplataforma: instalación, funcionamiento sin conexión sin salir del navegador ni depender de una tienda de aplicaciones ni de un runtime embebido como Electron.

11. Lista de Cotejo

Reviso, a manera de autoevaluación, que el documento cumpla con todos los puntos solicitados para esta actividad de cierre:

ELEMENTO SOLICITADO	CUMPLE	OBS.
Portada con formato oficial UTP	Sí	
Índice / tabla de contenido	Sí	
Introducción	Sí	
Definición de qué es una Aplicación Web Progresiva	Sí	
Características	Sí	
Ventajas y desventajas	Sí	
Herramientas de desarrollo	Sí	
Ejemplos con su URL	Sí	6 casos reales con URL y resultados reportados
Evidencias de identificación de características	Sí	3 evidencias reales sobre squoosh.app, con descripción
Casos de uso recomendados	Sí	Agregado para orientar cuándo conviene este enfoque
Comparativa contra los enfoques del curso	Sí	Agregado como cierre de la materia
Conclusión	Sí	
Referencias	Sí	

12. Conclusión

Explorar el material de lectura sobre Aplicaciones Web Progresivas me dejó claro que este enfoque no compite con los seis que documenté antes, sino que los conecta: una PWA es, en esencia, una aplicación web (Subproducto No. 2) a la que se le agregan un manifiesto y un service worker para acercarla a lo que ofrecen las apps nativas, móviles, de escritorio o multiplataforma, sin salir del navegador ni depender de una tienda de aplicaciones. Los casos reales que documenté Starbucks, Twitter Lite, Pinterest, AliExpress, Uber y Forbes muestran que esa ganancia no es solo teórica: se traduce en apps mucho más ligeras, más rápidas en redes lentas, y con mejoras medibles en sesiones, conversión y tiempo de uso.

La sección de casos de uso recomendados me dejó claro también que la PWA no es la respuesta correcta para todo: sigue siendo una decisión de arquitectura, no una mejora automática, y hay escenarios juegos con exigencias gráficas altas, apps que dependen de APIs de seguridad muy específicas del dispositivo donde alguno de los otros seis enfoques que documenté durante el curso sigue siendo la mejor opción.

Con este séptimo y último subproducto cierro el panorama completo que construí a lo largo de la materia: nativo (Wear OS), web (MediCore), escritorio (TaskDesk), móvil (PediatriTrack Global), multiplataforma (ExpenseTrack Lite), orientado a servicios (HabitPulse) y ahora web progresivo. Viéndolos en conjunto, ningún enfoque es superior en abstracto: cada uno resuelve una combinación distinta de necesidades rendimiento, alcance, costo, acceso al hardware, dependencia de internet y en un proyecto real es común combinar varios, como una PWA que además consume uno o varios servicios propios, de la misma forma en que HabitPulse podría convertirse algún día en el backend de una interfaz web instalable.

Referencias

- Wikipedia contributors. (2026). *Progressive web app*. Wikipedia.
https://en.wikipedia.org/wiki/Progressive_web_app
- Mozilla Developer Network. (2026). *Progressive web apps (PWAs)*. MDN Web Docs.
https://developer.mozilla.org/es/docs/Web/Progressive_web_apps
- Google. (2026). *Progressive web apps*. web.dev. <https://web.dev/explore/progressive-web-apps>
- Google. (2026). *Workbox* [Documentación]. Chrome for Developers.
<https://developer.chrome.com/docs/workbox>
- Google. (2026). *Lighthouse* [Documentación]. Chrome for Developers.
<https://developer.chrome.com/docs/lighthouse/overview>
- Microsoft. (2026). *PWA Builder*. <https://www.pwabuilder.com/>
- MobiLoud. (2026). *Progressive web app examples: 30 PWAs worth studying*.
<https://www.mobiloud.com/blog/progressive-web-app-examples>
- Vite PWA. (2026). *vite-plugin-pwa* [Documentación]. <https://vite-pwa-org.netlify.app/>
- Russell, A. (2015, junio 15). *Progressive web apps: Escaping tabs without losing our soul*. Infrequently Noted. <https://infrequently.org/2015/06/progressive-apps-escaping-tabs-without-losing-our-soul/>
- W3C. (2026). *Web app manifest*. <https://www.w3.org/TR/appmanifest/>
- Google Chrome Labs. (2026). *Squoosh* [Aplicación web]. <https://squoosh.app/>